Sistemas de Redes y Servicios de Internet

Laboratorio OpenSSH

Análisis de conexiones, reenvío de puertos y autenticación

Jorgr Arévalo
Jon González

2 de octubre de 2026

Contents

1 Introducción **2**

2 Entorno y preparación **2**

3 Comparación entre Telnet y SSH con Wireshark **2**

4 Ejercicio 1: conexión SSH en modo verbose **2**
- 4.1 Procedimiento . . . 2
- 4.2 Clasificación de los mensajes . . . 3
- 4.3 Resultado . . . 3

5 Ejercicio 2: comprobación de la clave del host **3**

6 Ejercicio 3: reenvío local de puertos **4**
- 6.1 Configuración del túnel . . . 4
- 6.2 Prueba desde Firefox . . . 4

7 Ejercicio 4: autenticación mediante clave pública **6**

8 Conclusiones **6**

1 Introducción

El objetivo de este laboratorio es estudiar el funcionamiento de OpenSSH y compararlo con una conexión Telnet. En esta entrega se documentan únicamente las pruebas realizadas hasta el momento: preparación de Telnet, observación de los puertos, análisis de una conexión SSH en modo verbose, reenvío local de puertos y autenticación mediante clave pública.

Los ejercicios de gestión de claves con `ssh-agent` y de una funcionalidad adicional de OpenSSH no se incluyen, porque todavía no se dispone de resultados o capturas de esas pruebas.

2 Entorno y preparación

Las pruebas se realizaron en Debian GNU/Linux, en la máquina `bocata-jamon`, utilizando los usuarios `user1` y `user2`. La versión observada fue OpenSSH 10.0p2, sobre el núcleo `6.12.107+deb13-amd64`.

Se instalaron Telnet y su servidor:

```
user1@bocata-jamon:~$ sudo apt install telnet telnetd
```

Después se descomentó en `/etc/inetd.conf` la siguiente línea:

```
telnet stream tcp nowait root /usr/sbin/tcpd /usr/sbin/telnetd
```

Finalmente se reinició el servicio:

```
user1@bocata-jamon:~$ sudo systemctl restart inetd
```

La salida de `ss -tan` confirmó que SSH escuchaba en el puerto 22 y Telnet en el puerto 23:

```
user1@bocata-jamon:~$ ss -tan
LISTEN 0 128 0.0.0.0:22 0.0.0.0:*
LISTEN 0 10  0.0.0.0:23 0.0.0.0:*
LISTEN 0 128 [::]:22    [::]:*
user1@bocata-jamon:~$
```

3 Comparación entre Telnet y SSH con Wireshark

Se preparó Wireshark para escuchar en la interfaz de loopback. La conexión Telnet transmite la sesión sin cifrado, por lo que el contenido de la conversación puede observarse al seguir el flujo TCP. SSH, en cambio, negocia los algoritmos criptográficos y, después del intercambio de claves y de `SSH2_MSG_NEWKEYS`, protege el contenido de la sesión.

En las pruebas se utilizó la opción de Wireshark *Follow Stream* sobre el flujo correspondiente a SSHv2. Las capturas de Wireshark se añadirán a esta sección cuando estén disponibles; no se simulan aquí resultados que no han sido aportados.

4 Ejercicio 1: conexión SSH en modo verbose

4.1 Procedimiento

Se realizaron las siguientes conexiones desde `user1` hacia `user2`:

```
user1@bocata-jamon:~$ ssh -v user2@localhost
user1@bocata-jamon:~$ ssh -vv user2@localhost
```

El cliente resolvió `localhost` como `::1` y conectó con el puerto 22. En ambos casos la conexión terminó correctamente después de introducir la contraseña de `user2`.

4.2 Clasificación de los mensajes

Los mensajes observados se pueden clasificar según los componentes de SSH:

Componente	Mensajes y significado
Transporte SSH	El cliente y el servidor anuncian SSH-2.0. Se seleccionan `mlkem768x25519-sha256`, la clave de host `ssh-ed25519` y el cifrado `chacha20-poly1305@openssh.com`.
Intercambio de claves	`SSH2_MSG_KEXINIT` y `SSH2_MSG_KEX_ECDH_REPLY` muestran la negociación de algoritmos y el intercambio de material criptográfico.
Clave del servidor	El cliente comprueba que la clave ED25519 coincide con la registrada para `localhost` en `/.ssh/known_hosts`.
Autenticación	El servidor anuncia los métodos `publickey,password`. Como no había una clave utilizable, el cliente continúa con contraseña.
Sesión SSH	Tras `SSH2_MSG_NEWKEYS` se abre el canal de sesión, se solicita una pseudo-terminal y se inicia la shell remota.

El modo `-vv` añade información más detallada sobre las propuestas de algoritmos, el canal 0, la solicitud de pseudo-terminal y la confirmación de la shell. Al cerrar la sesión se observan el cierre del canal y el código de salida 0.

4.3 Resultado

La conexión se estableció correctamente. El análisis muestra que SSH separa la negociación criptográfica, la autenticación y la apertura de la sesión. La contraseña se introduce después de establecer el canal protegido, por lo que no queda expuesta como texto legible en la red.

5 Ejercicio 2: comprobación de la clave del host

En esta prueba se compararon dos estados del cliente SSH: `ssh1`, que corresponde a la versión configurada, y `ssh2`, que corresponde a la versión instalada de cero.

Las principales diferencias, en orden de aparición al utilizar la opción `-v`, fueron las siguientes:

- Al inicio, `ssh1` lee la configuración en `/etc/ssh/ssh_config`, en `/etc/ssh/ssh_config.d/20-systemd-ssh-proxy.conf` y aplica la configuración.
- `ssh2` muestra un par de veces un error de `fopen` al intentar abrir `.ssh/known_hosts`.
- `ssh1` comprueba que "localhost" es un host conocido y que se corresponde con la clave ED25519.
- `ssh2`, en ese mismo punto, busca los ficheros que contienen firmas de hosts y, al no encontrarlos, determina que no se puede comprobar la autenticidad del host.
- `ssh2` avisa al usuario de que no se conoce la clave y pregunta si quiere continuar.
- `ssh2` avisa de que el soporte de SELinux está deshabilitado.
- `ssh2` muestra un aviso de que el host "localhost" se ha añadido permanentemente a la lista de `known_hosts`.

- `ssh1` avisa de que no hay ninguna clave del servidor nueva o revocada y carga las variables de entorno `COLORTERM` y `LANG`.
- `ssh2`, en ese punto, avisa de que ha aprendido nuevas claves del host y que las ha guardado.

Al intentar conectarse, como ya no se tenían guardadas las claves del host, SSH avisó de que no se podía determinar la autenticidad del host, mostró la clave ED25519 y preguntó si se quería continuar. Al responder afirmativamente, guardó la clave.

6 Ejercicio 3: reenvío local de puertos

6.1 Configuración del túnel

Desde la máquina local se ejecutó:

```
user1@bocata-jamon:~$ ssh -L 12000:ehu.eus:443 user1@SRDSI
```

La opción `-L` crea un puerto local, el 12000, cuyo tráfico se envía a través de SSH hacia `ehu.eus:443`. El túnel se comprobó mediante:

```
user1@bocata-jamon:~$ sudo netstat -tanp | grep 12000
tcp 0 0 127.0.0.1:12000 0.0.0.0:* LISTEN 324823/ssh
tcp6 0 0 ::1:12000       :::*              LISTEN 324823/ssh
```

La salida muestra que un proceso `ssh` estaba escuchando en `127.0.0.1:12000` y en `::1:12000`.

6.2 Prueba desde Firefox

Se intentó acceder a `localhost:12000` desde Firefox antes y después de establecer el túnel.

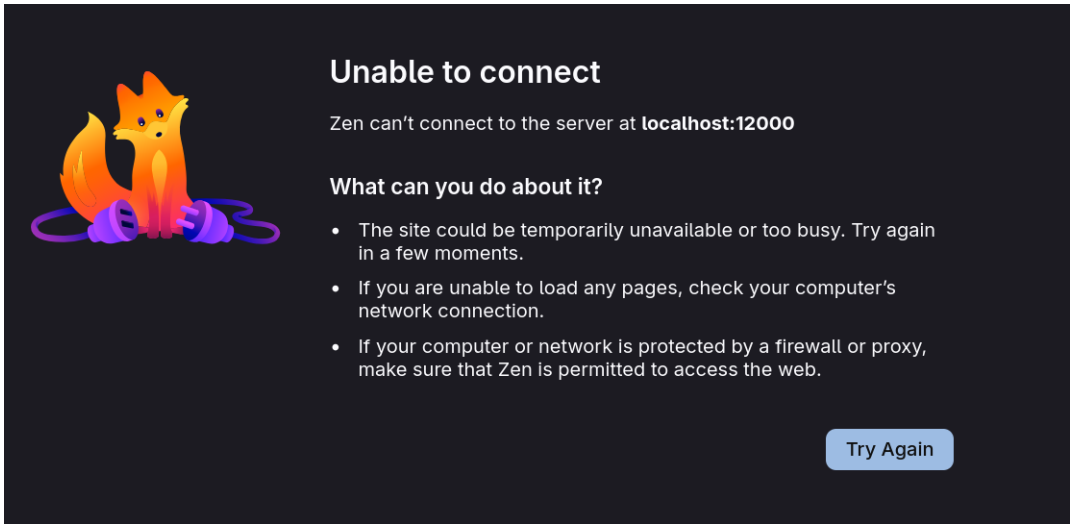


Figure 1: Intento de conexión a `localhost:12000` sin el túnel.

Sin el túnel, Firefox mostró "Unable to connect" y no pudo conectarse al servidor en `localhost:12000` (figura 1).

Con el túnel activo, Firefox recibió una respuesta HTTPS, pero mostró el error `SSL_ERROR_BAD_CERT_DOMAIN`. La pantalla indica que el certificado es válido para nombres de `ehu.eus`, no para `localhost:12000`. Por tanto, la captura demuestra que se alcanzó el servicio HTTPS remoto a través del reenvío, pero la navegación no terminó con una página cargada debido a la comprobación del nombre del certificado.

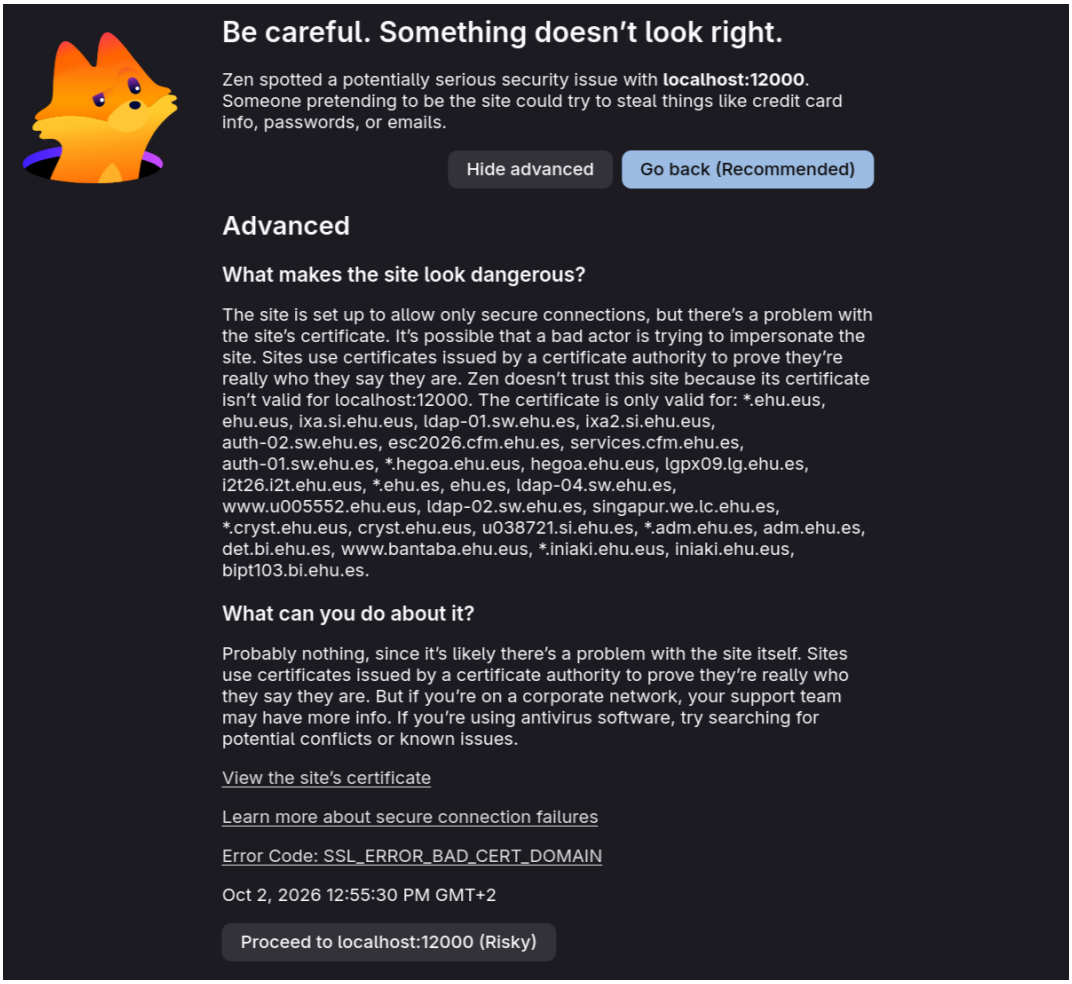


Figure 2: Intento de conexión a `localhost:12000` con el túnel.

7 Ejercicio 4: autenticación mediante clave pública

Primero se comprobó que la conexión de `user1` a `user2` pedía la contraseña:

```
user1@bocata-jamon:~$ ssh user2@localhost
```

Después se generó para `user1` un par de claves ED25519:

```
user1@bocata-jamon:~$ ssh-keygen
```

La clave privada se guardó en `/.ssh/id_ed25519` y la pública en `/.ssh/id_ed25519.pub`. La clave pública se copió a la cuenta remota con:

```
user1@bocata-jamon:~$ ssh-copy-id user2@localhost
```

Este comando pidió la contraseña para instalar la clave en `user2/.ssh/authorized_keys`. Después, al ejecutar de nuevo `ssh user2@localhost`, la conexión se realizó sin pedir la contraseña de la cuenta remota. Solo se copió la clave pública; la clave privada permaneció en `user1`.

8 Conclusiones

Las pruebas realizadas muestran la diferencia principal entre Telnet y SSH: Telnet no cifra la sesión, mientras que SSH negocia algoritmos criptográficos, verifica la identidad del servidor y protege la autenticación y la sesión.

También se comprobó el reenvío local de puertos. Sin el túnel, el puerto local 12000 no aceptaba la conexión. Con el túnel, la petición llegó al servicio HTTPS de `ehu.eus`; el error observado se debió a que el certificado remoto no corresponde al nombre `localhost`. Finalmente, la autenticación mediante clave pública permitió conectarse sin introducir de nuevo la contraseña de `user2`.